FINAL WRITE UP

INTRODUCTION

The foundation of any machine learning architecture is fundamentally governed by the quality, diversity, and scale of its datasets. Datasets serve as the primary blueprint from which a network learns to map complex mathematical distributions and generalize to real-world patterns. Managing high-fidelity real-world datasets presents a significant engineering bottleneck, as their huge physical size creates massive space consumption and download strain that:

a. Chokes the local storage
b. Stalls network bandwidth
c. Slows down rapid prototyping cycles on cloud environments.

To bypass these hardware and logistical constraints, we use data that is generated using an algorithm, which is similar to real-world data. These generated data streams are called synthetic data. They don't contain storage overhead or privacy risks. Synthetic data can be compiled on the fly with custom parameter variations, allowing developers to precisely manipulate variables like vocabulary complexity or sequence length to stress-test model boundaries, track exact loss convergence, and isolate hardware execution limits under perfectly controlled conditions. Yet, whether an architecture relies on real or synthetic data, its performance ultimately depends on the efficiency of its data pipelines. These specialized software engines act as the high-speed assembly line of the training system, transforming, batching, and sharding raw data arrays smoothly from storage into accelerator registers. This ensures that powerful computational backends like GPUs and TPUs never sit idle waiting for math to process.

DENSE AND MOE MODELS

The processing efficiency and performance of a machine learning model depend on the model's structural parameters which govern the network's learning capacity. Scaling these parameters allows the model to map deeper complexities within the dataset, but exponentially increases both the computational load and the hardware memory requirements. Traditionally, this scaling has been achieved through dense architectures—where the nature of the model dictates that every single parameter is universally activated for every passing token. This results in exhaustive, full-matrix computations that heavily bottleneck accelerator backends.

In contrast, a Mixture of Experts (MoE) architecture fundamentally changes this working mechanism by decoupling total memory capacity from active computational cost. An MoE

model retains a massive total parameter count residing statically in memory to ensure high generalized knowledge, but only activates a fraction of the network at a time. Mathematically, while a dense model optimizes a standard cross-entropy loss by multiplying data across its entire parameter space, the MoE introduces a routing mechanism. A linear layer followed by a Softmax gating function that calculates a probability distribution for each token, multiplies the data with only the highest probability "top-k" expert pathways. This specialized routing drastically reduces the active matrix multiplications required per step, though it necessitates a dual-loss optimization strategy. Pairing the main task loss with an auxiliary load-balancing mathematical penalty ensures that tokens are distributed evenly across all experts. This allows the MoE to execute on resource-constrained backends at a high speed without sacrificing the intelligence inherent to larger models.

BACKEND AND SIZE DIFFERENCES

On a practical hardware level, the core contrast lies in how their physical size dictates their compatibility with different backends. The dense Qwen model maintains a rigid footprint of roughly 800 million parameters that are permanently active, meaning every backend—whether it is a standard sequential CPU or a parallelized GPU—must execute massive, unyielding matrix math for every calculation, which easily leads to Out-of-Memory crashes on restricted systems. Conversely, while the DeepSeek Mixture of Experts model holds a slightly larger total structural size of nearly 900 million parameters in static storage to maintain a deep knowledge base, it isolates its active computational workload to a 265 million parameters per step. This architectural difference allows the MoE model to bypass the memory limitations of smaller backends like the CPU, run efficiently on single GPUs, and scale perfectly across advanced multi-core backends like TPUs, where frameworks like JAX can seamlessly share the lighter active workload across multiple processors simultaneously.

TABLE 1: ARCHITECTURE COMPARISON

METRIC	QWEN 0.8B	DEEPSEEK 0.9B
Model nature	Standard dense	Sparse (Mixture of Experts)
Total parameters	800 million	890 million
Active parameters	800 million (100% active)	265 million (30% active)
Total layers	24	16

Global hidden state	1536/2048	1024
Intermediate state	4096	1024
Key/Value heads	Standard multi-head	8 (Grouped-query attention)
Total experts per layer	N/A	16
Shared experts (always active)	N/A	2
Routed experts	N/A	14
Routing strategy	N/A	Top-2 gating (top_k = 2)

TABLE 2: WORKING NATURE AND MATHEMATICAL CONCEPTS

METRIC	QWEN 0.8B	DEEPSEEK 0.9B
Forward pass equation	y = FFN (x)	$y = \sum_{i \in \text{Top-k}} G(x)_i E_i(x) + \sum_j E_{shared}(x)$
Optimization objective	Single Task Loss (Standard Softmax Cross-Entropy)	Dual Objective (Task Loss + Auxiliary Load Balancing Penalty)
Load balancing	N/A	$L_{balance} = N \sum_{i=1}^{N} f_i \cdot P_i$ (Penalizes the router for favoring specific experts).
Matrix sparsity	0% (Dense matrix multiplication for all the tokens - W.x)	Highly sparse (Multiplication is masked by 0 for all unselected experts)
Memory vs compute	Tightly Coupled (Storing parameters costs the same as computing them)	Decoupled (High storage capacity, but low active computational cost)
Attention matrix scaling	Standard Multi-Head Attention (1 Q-head per 1 KV-head)	Grouped-Query Attention (Multiple Q-heads share 1

		KV-head matrix to save memory)

TABLE 3: METRICS COMPARISON

METRIC	QWEN 0.6B (CPU)	DEEPSEEK 0.9B (CPU)	QWEN 0.8B (GPU)	DEEPSEEK 0.9B (GPU)	QWEN 0.8B (TPU)	DEEPSEEK 0.9B (TPU)
Throughput tokens/sec	28.01	54.43	967.41	1360.43	24667.31	1372.01
Average step time	18.6514	9.4068	0.54	0.3764	0.0212	0.3732
Total execution time	929s	514.04s	44s	57.08s	112s	57.75s

NOTE: Since the type of metrics reported for both the models are not entirely the same, they are separately shown in the outputs directory.

INTERPRETATIONS AND IMPORTANT LEARNINGS

The core architectural insights and inferences made from stress-testing both the dense and sparse models across different compute backends is explained here

1. The Core Efficiency Frontier: Compute vs. Memory
The primary inference from these benchmarks is the empirical proof of the **memory-compute decoupling** offered by Mixture of Experts (MoE).

- **The Intelligence-to-Compute Ratio:** Traditional dense models bind capacity directly to computation. To make a dense model smarter, you must add parameters, which automatically forces every hardware backend to execute more floating-point operations (FLOPS) per token. **MoE breaks this linear scaling law.**
- **Dynamic Parameter Allocation:** The DeepSeek model proves that a network can maintain the broad knowledge footprint of an ~890M parameter model in storage while executing with the agility and speed of a ~265M parameter model. The computational footprint per token remains flat even as the total expert capacity grows.

2. Behavioral Mapping of Backends

Each hardware backend exhibited a distinct profile that reveals its architectural suitability for dense vs. sparse workloads:

- **CPU (The Memory Bottleneck):** Sequential processing architectures are fundamentally ill-suited for dense models above 500M parameters due to XLA compilation overhead and optimizer state inflation. However, the MoE structure is a viable architecture for constrained CPU environments because its **active tensor sizes are small** enough to process within standard system memory limits, avoiding hard out-of-memory terminations.
- **GPU (The Bandwidth Bottleneck):** On a single accelerator like the T4, the MoE model runs rapidly but faces a strict VRAM bandwidth wall. Because the GPU must constantly hold the inactive weights of all 14 non-selected experts, the performance bottleneck shifts from raw calculation speed to the speed at which the GPU can stream weight matrices into its active registers.
- **TPU (The Scaling Ideal):** Multi-core tensor processors combined with JAX execution templates represent the optimal environment for MoE. Through framework-level sharding **(jax.pmap)**, the compute workload of the **active experts is distributed** perfectly across independent processing cores. This allows the system to scale throughput exponentially when processing larger batch sizes.

3. The Mechanics of Learning in Sparse Systems

Observing the loss curves and token routing distributions under training constraints provided critical insights into how sparse networks train compared to dense networks:

- **Router Vulnerability:** Dense models distribute learning evenly across all weights via backpropagation. In contrast, an MoE model's performance relies entirely on the stability of its routing layer. If the router initializes poorly, it can create a runaway feedback loop where it continuously feeds tokens to a single expert, leading to expert collapse and an immediate halt in structural learning.
- **The Mathematical Necessity of Regularization:** The auxiliary load-balancing loss is not just an optimization feature; it is an absolute structural dependency. Without the explicit mathematical penalty (L_balance), the sparsity mechanism breaks down completely, causing the model to default to an inefficient, pseudo-dense state where hardware resources are completely misallocated.

UNEXPECTED FINDINGS ALONG THE WAY: THE BEHAVIOUR OF THE MODEL, THE CONSTRAINTS ON RESOURCES AND ENVIRONMENTS

1. The Disconnect Between Theoretical Math and Real-World Speedup

The underlying math of our MoE model dictating a reduction from 890M total parameters to roughly 265M active parameters suggests a theoretical compute savings of ~70% per forward pass. However, raw execution time on the T4 GPU did not scale linearly with a perfect 70% decrease.

- **The Routing Bottleneck:** It was discovered that routing tokens is not mathematically "free." Evaluating a linear layer, applying a Softmax gating function, sorting the Top-2 choices, and then dynamically indexing arrays introduces a distinct operational overhead.
- **Memory Bandwidth vs. Compute Power:** On smaller hardware like a single T4 GPU, execution speed is frequently limited by memory bandwidth (moving data in and out of VRAM) rather than raw FLOPS (floating-point operations per second). Because the GPU still had to hold the full 890M parameters in memory, the constant swapping of expert weights slowed down the absolute execution time, yielding an empirical **2.5x speedup** rather than the **theoretical 3.3x speedup**.

2. Behavior Under Synthetic Data Streams

Using synthetic datasets to benchmark the model provided a controlled laboratory environment, but it exposed an interesting phenomenon regarding how the routing engine learns.

- **The "Perfect Distribution" Illusion:** Because the synthetic data generated token sequences with a highly predictable, uniform distribution, the **router learned to distribute** the workload across all 14 routed experts with near-perfect equality almost instantly.
- **The Auxiliary Loss Vanishing Act:** Due to this uniform distribution of input data, the Load Balancing Loss (L_balance) dropped to near-zero within the first 10 steps. While ideal for benchmarking throughput, this revealed that: **synthetic data fails to properly stress-test** the router's ability to handle the "heavy-tail" language biases found in **real-world human speech**, where certain words or topics would naturally cause a massive bottleneck on a single expert.

3. JAX Compilation and Software Engineering Principles

Porting a dynamic routing architecture to a strictly static compilation framework like JAX led to several critical engineering errors that had to be resolved during implementation.

- **The Intermediates Collection Trap (KeyError: 'load_balance_loss'):** Flax standard functional blocks expect layer outputs to pass straight up the transformer block. Because the load-balancing loss is calculated **inside** the MoE layer but needed at the **global training loop** level, capturing it natively caused JAX trace errors. We had to explicitly harvest the loss using **Flax's sow mechanism** to append it into the intermediates collection, a structural quirk completely absent in conventional dense networks.

- **Dataclass vs. FrozenDict Trace Collisions:** During configuration initialization, trying to pass the hyperparameter settings as a raw Python dictionary into the JAX-traced model functions threw immediate compilation exceptions. JAX requires **all inputs** to compiled functions to be explicitly **hashable static types or arrays.** This forced the migration to an explicit, strongly typed @dataclass format to ensure the compilation tracing engine could freeze the structural dimensions of the 16 experts.
- **The Silent CPU Compiler Crash:** The most striking hardware anomaly was how the two models failed on the CPU. The MoE model processed slowly but safely on the CPU. However, **the dense model triggered a sudden, silent termination of the Google Colab backend.** The error was **not caused by executing the matrix operations, but rather by the XLA compilation phase**. The static graph computation for an 800M dense parameter framework combined with the memory states of the AdamW optimizer completely exhausted system RAM before a single step was ever run.

FINAL SUMMARY

The building of a model highly depends on the nature of datasets. The quality of a dataset will determine the performance of the model since the model is trained on the dataset. To push the limits of a model, we need to use synthetic data which is helpful to make inferences to a good extent. However, we can't fully rely on these datasets but they are safe enough for a good interpretation before using a real dataset. Depending on the use case and our requirements, we need to choose the right trade-off for the model and identify the right hyperparameters for the same. While building a model, the right architecture should always have a design where we split the files by giving them the right responsibilities. The most important aspect of a model is its configurations and all the major configurations should be done in a separate file that can be modified later and do not require changes in every file. The model should be universal and run on almost all the hardware types. In this regard, MoE is a great choice due to its working mechanism and routing. Dense models need heavy computational power and are sometimes not compatible with CPUs if the parameters exceed 0.6B. For real-world datasets, the hyperparameter tuning should be done with careful calculations since it can waste a lot of computational resources if not stress-tested properly. The usage of resources is very important since cloud environments have resource usage policies and limitations, and we need to use them efficiently.